Lesson 7 – Over-Privileged Function

A Lambda compromise becomes much worse because the function role has more AWS permissions than it actually needs, so the attacker can pivot from one function into unrelated backend data.

1. Goal and Vulnerability Summary

The goal of this lesson was to examine whether the Lambda function **DVSA-SEND-RECEIPT-EMAIL** was running with more AWS permissions than required for its intended task. In this case, the function was supposed to process receipt-related operations, but its execution role granted broader access than necessary. In addition, the function could expose sensitive runtime information through logging. This combination increases the blast radius of the vulnerability because if the function is abused, the attacker may gain access to unrelated backend resources, as demonstrated in this project through stolen temporary credentials reused to enumerate backend Lambda functions.

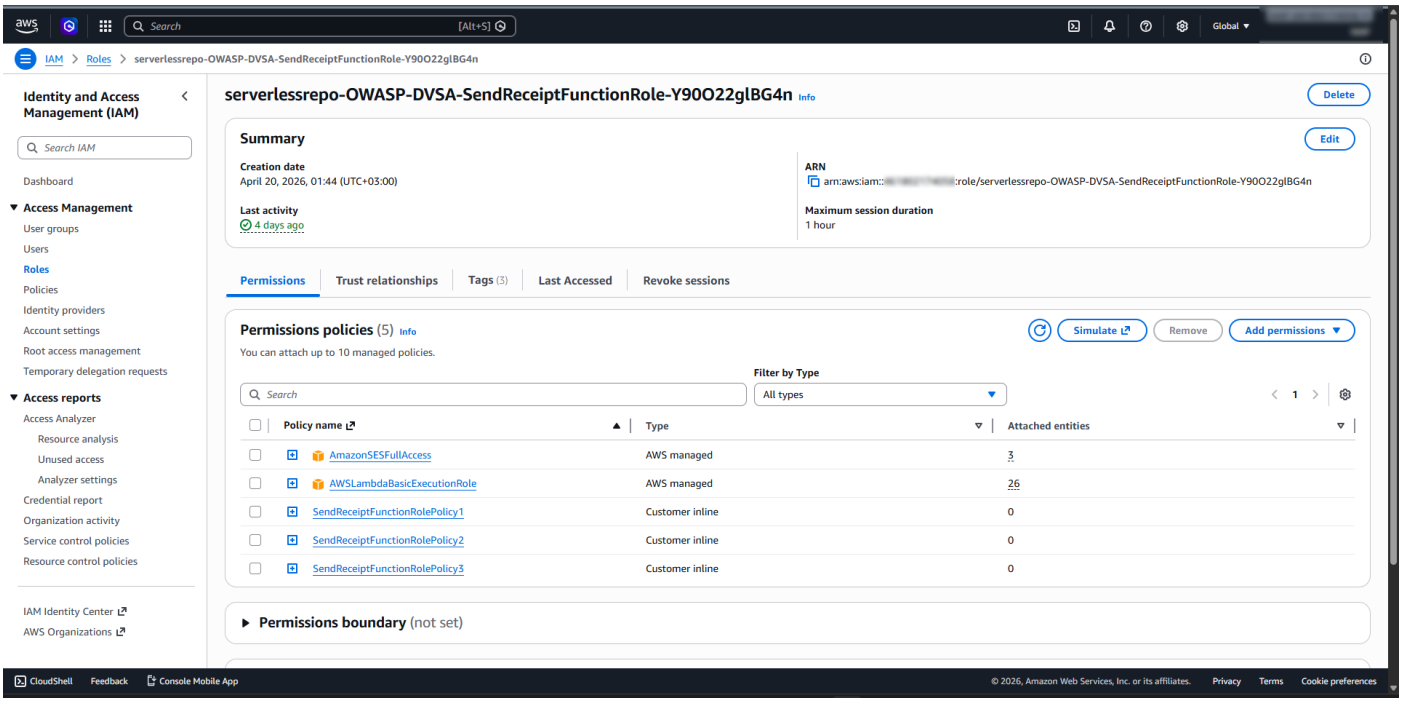

Figure 1: Screenshot #1 - Execution Role Policies

2. Why This Works / Root Cause

The vulnerability exists because the Lambda execution role violates the principle of least privilege. In particular, the role included unnecessary DynamoDB permissions that were not required for simple receipt-email processing. The issue became more dangerous when environment variables were printed into CloudWatch logs, exposing temporary AWS credentials associated with the function. As a result, a flaw in one function could potentially lead to broader AWS access than intended.

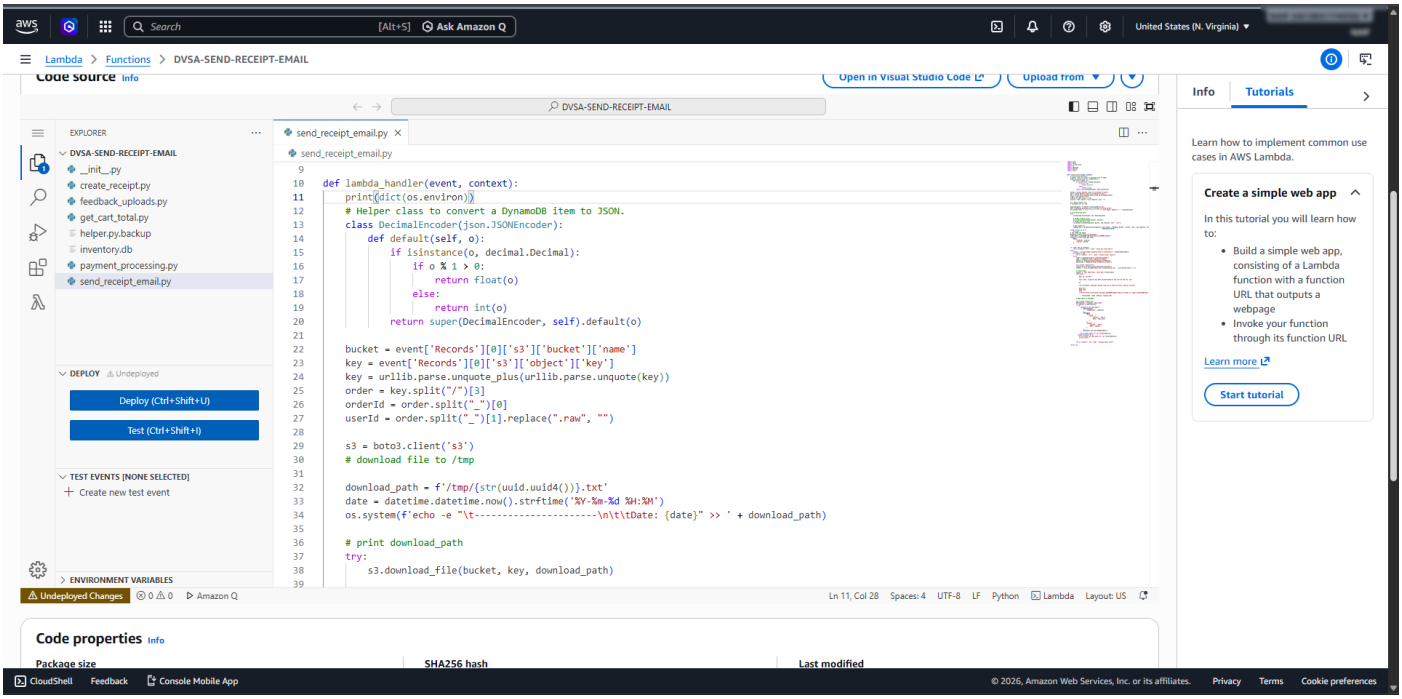

Figure 1: Screenshot #2 - Leaking Credentials Code

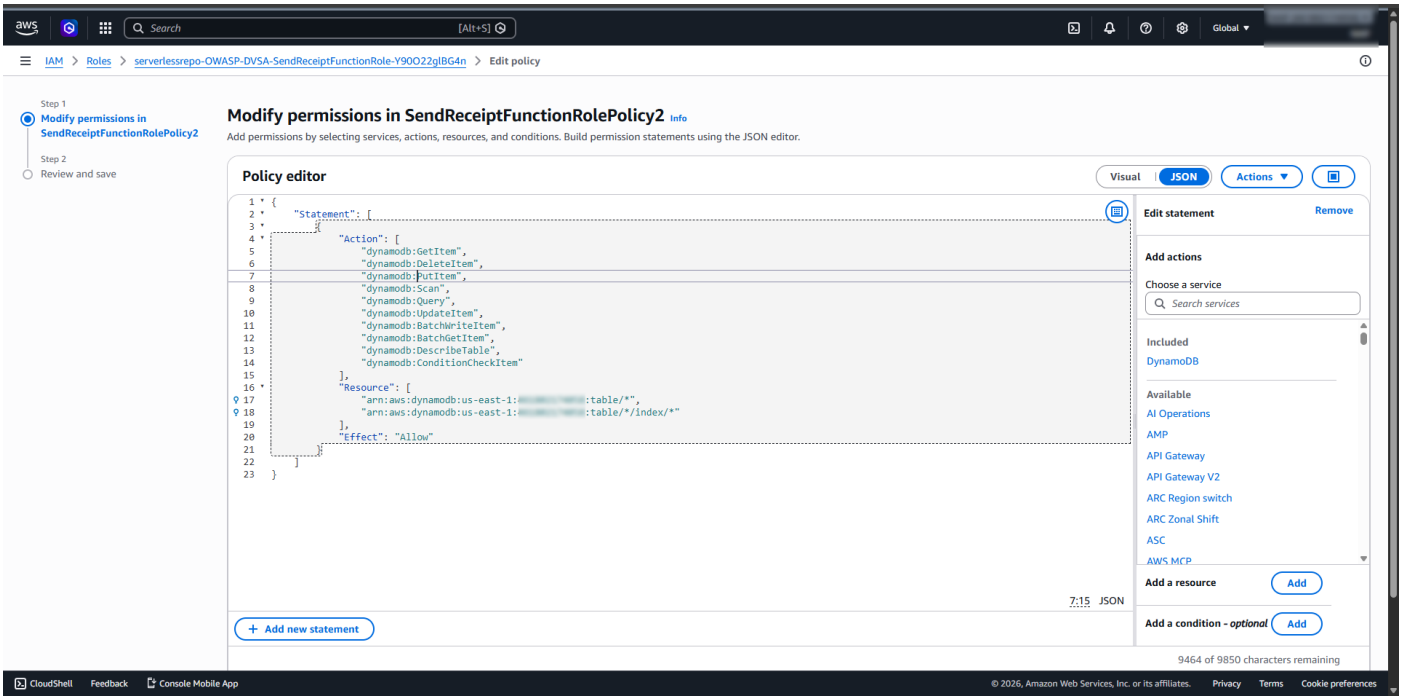

Figure 2: Screenshot #5 - Overbroad Policy2 Before Fix

3. Environment and Setup

DVSA was deployed in the **us-east-1** region in a controlled non-production AWS account. Two test users were created in the application and sample orders were placed to generate normal application data. The vulnerable function used in this lesson was **DVSA-SEND-RECEIPT-EMAIL**, and its trigger path involved the receipts S3 bucket. A controlled S3 object upload was used to invoke the function and observe its behavior through CloudWatch logs.

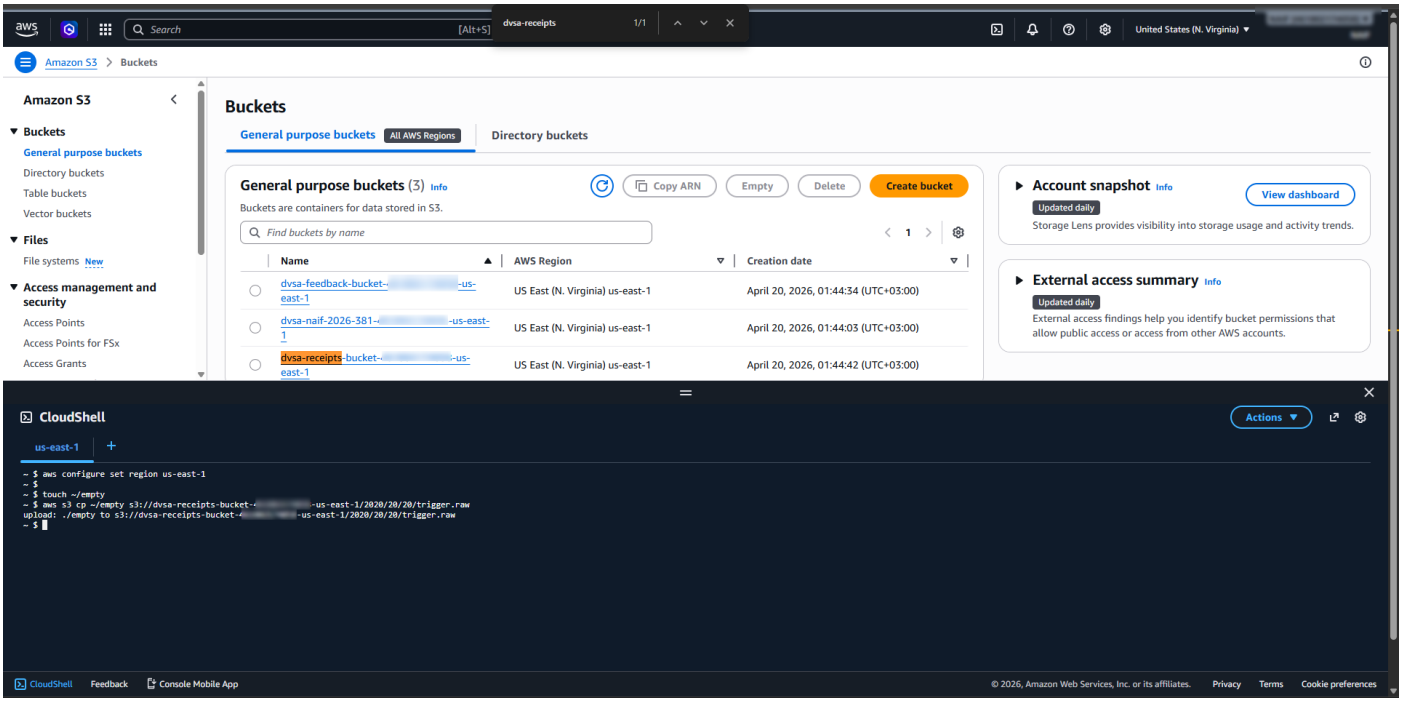

Figure 3: Screenshot #3 - S3 Trigger

4. Reproduction Steps

First, the execution role attached to the function was inspected and its policies were reviewed. Then, the function code in **send_receipt_email.py** was modified to print environment variables using **print(dict(os.environ))**. After deploying the modified function, a dummy file was uploaded to the receipts bucket in order to trigger the Lambda execution. CloudWatch logs were then inspected, and the runtime environment output showed sensitive values such as temporary AWS credentials. This confirmed that the function leaked sensitive information through logs. In addition, inspection of the role policies showed that the function had broader DynamoDB permissions than needed, which made the weakness more severe.

5. Evidence and Proof

The proof for this lesson came from multiple sources. First, the execution role attached to **DVSA-SEND-RECEIPT-EMAIL** showed that the function had several policies, including a policy with overly broad DynamoDB permissions. Second, the modified **send_receipt_email.py** code printed environment variables to CloudWatch logs. After triggering the function through the receipts S3 bucket, the CloudWatch output showed sensitive runtime values, including temporary AWS credentials such as **AWS_ACCESS_KEY_ID**, **AWS_SECRET_ACCESS_KEY**, and **AWS_SESSION_TOKEN**. This demonstrated that the function leaked sensitive information through logs. The role inspection and policy contents also showed that the function had broader DynamoDB access than required for receipt processing, which increased the severity of the issue.

While the original analysis began with the DVSA-SEND-RECEIPT-EMAIL function and its overbroad permissions, the full-chain credential exfiltration was reproduced through the vulnerable Order Manager path, which demonstrated the same over-privilege impact at the application level by allowing stolen temporary credentials to be reused against unrelated AWS resources.

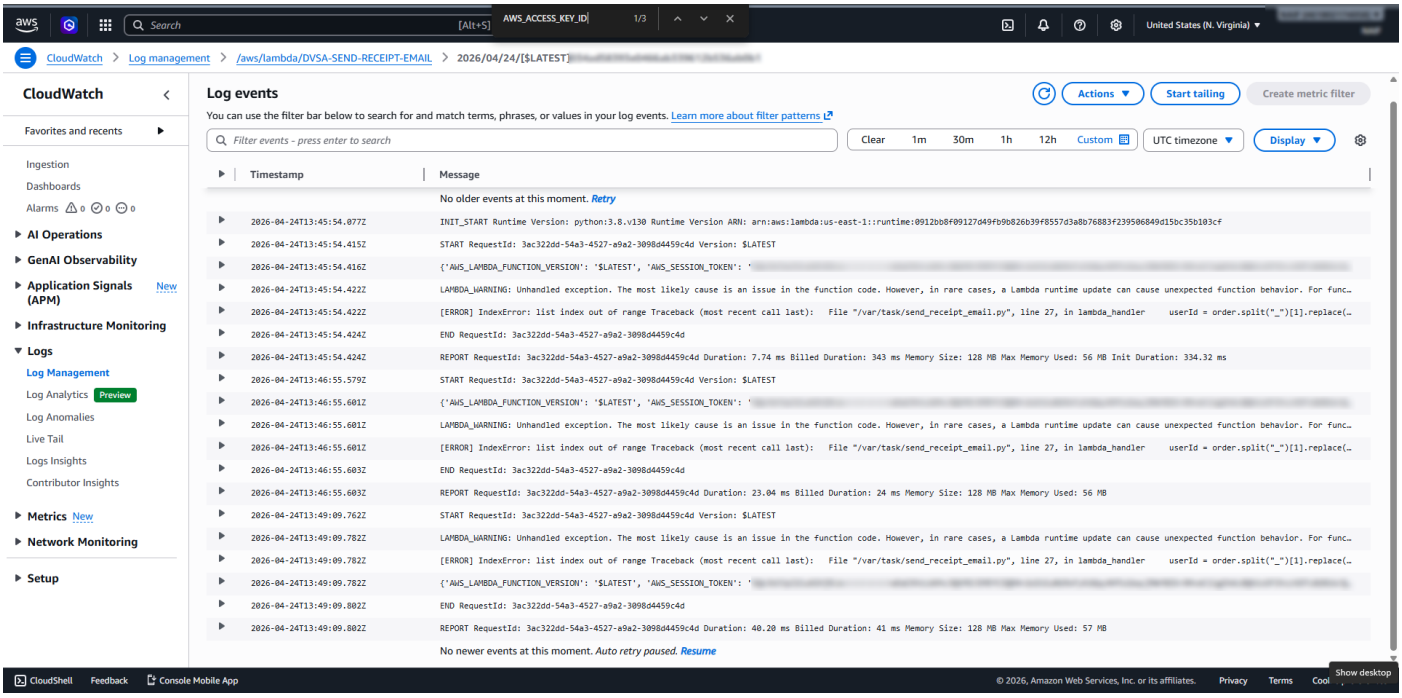

Figure 4: Screenshot #4 - Harvested Credentials

To demonstrate the full impact of the over-privileged role, the exfiltrated STS credentials (**AccessKeyId**, **SecretKey**, and **SessionToken**) were configured in a local **CloudShell** environment. By hijacking the Lambda's identity, it was possible to perform unauthorized actions that a receipt-processing function should never be allowed to do, such as listing all other Lambda functions in the account. This confirmed the 'full chain' exploit from initial leak to account-wide infrastructure enumeration.

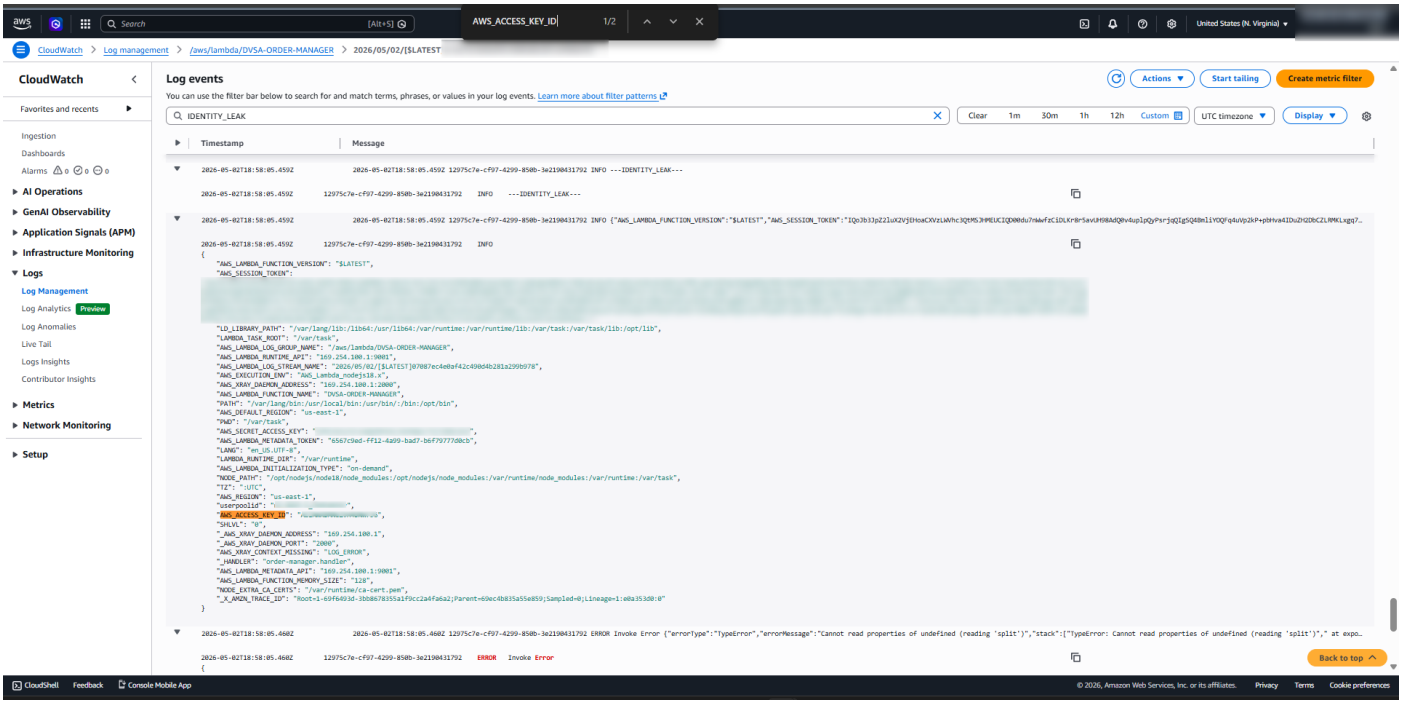

Figure 5: Exfiltration of temporary STS credentials from the vulnerable Order Manager path via the IDENTITY_LEAK payload.

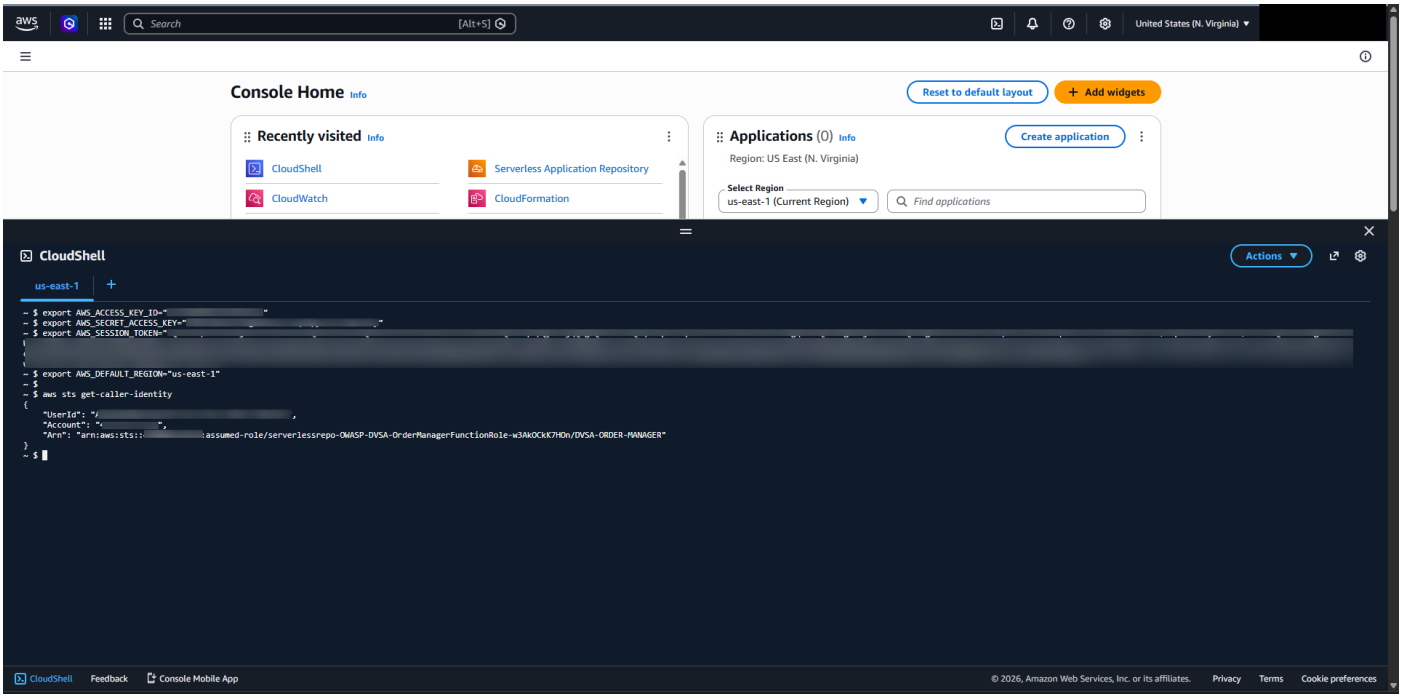

Figure 6: Verification of successful identity hijacking in CloudShell using the stolen STS session token.

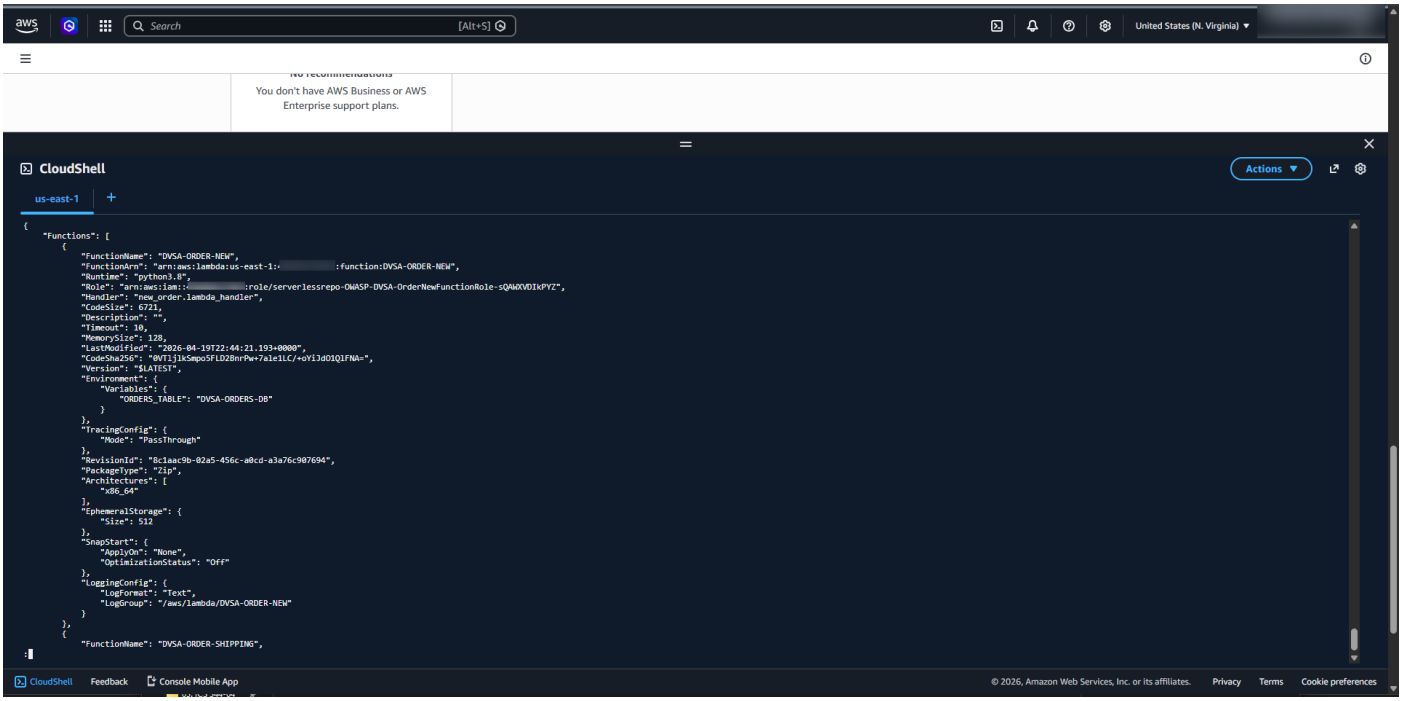

Figure 7: Proof of Over-Privilege—Using the hijacked role to list all backend infrastructure functions.

6. Fix Strategy / Probable Mitigation

The remediation had two main parts. First, the unsafe logging statement **print(dict(os.environ))** was removed from the Lambda code to stop exposing environment variables in CloudWatch logs. Second, the execution role was reduced to least privilege by modifying **SendReceiptFunctionRolePolicy2** and removing unnecessary DynamoDB actions such as scanning, querying, and broad data modification actions. After the fix, the function retained only the minimum permissions needed for its intended receipt-processing behavior.

7. Code / Config Changes

Two changes were applied during remediation. First, the temporary debugging statement **print(dict(os.environ))** was removed from **send_receipt_email.py** so that runtime environment variables would no longer be exposed in CloudWatch logs. Second, the IAM policy **SendReceiptFunctionRolePolicy2** was modified to reduce permissions to least privilege. Before the fix, the policy allowed broad DynamoDB actions such as **Scan**, **Query**, **PutItem**, **DeleteItem**, **UpdateItem**, **BatchWriteItem**, **BatchGetItem**, **DescribeTable**, and **ConditionCheckItem**. After remediation, the policy was reduced so that unnecessary DynamoDB actions were no longer allowed. This limited the function to only the permissions needed for its intended behavior.

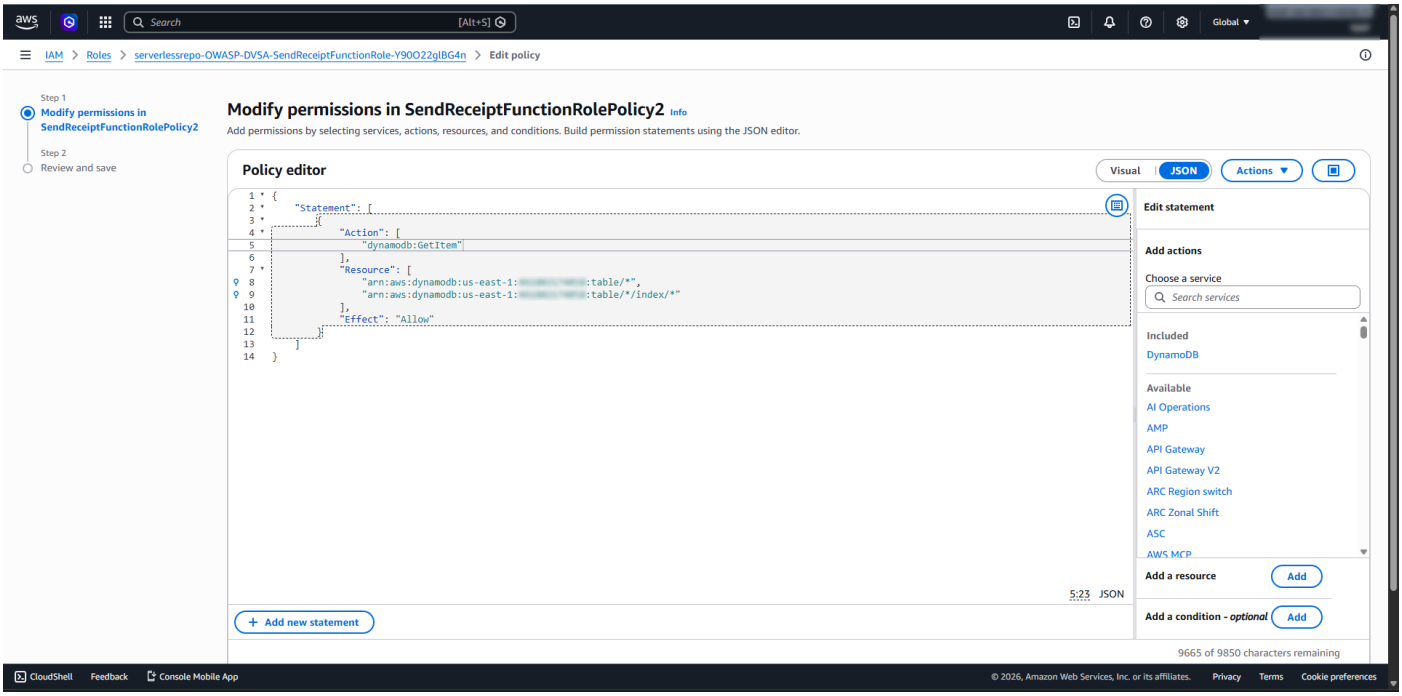

Figure 8: Screenshot #6 - Fixed Policy2 Least Privilege

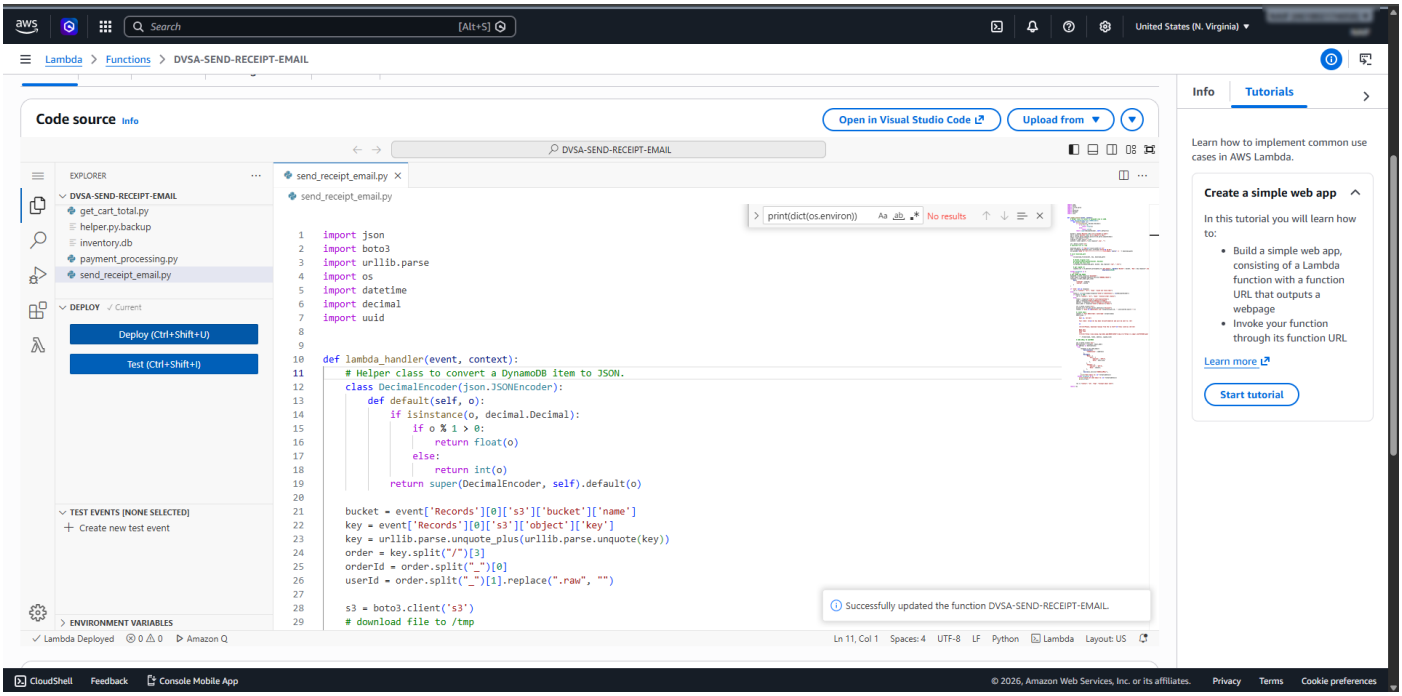

Figure 9: Screenshot #7 - Removed Credential Logging Code

8. Verification After Fix

After applying the fix, the function was deployed again and the same S3-trigger method was repeated. CloudWatch logs were checked to verify that sensitive values such as **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** no longer appeared in the log output. Then, IAM Policy Simulator was used to test **dynamodb:Scan** after the policy change, and the result showed that **dynamodb:Scan was denied**. This confirmed that the excessive privilege had been removed and that the sensitive logging issue was also fixed.

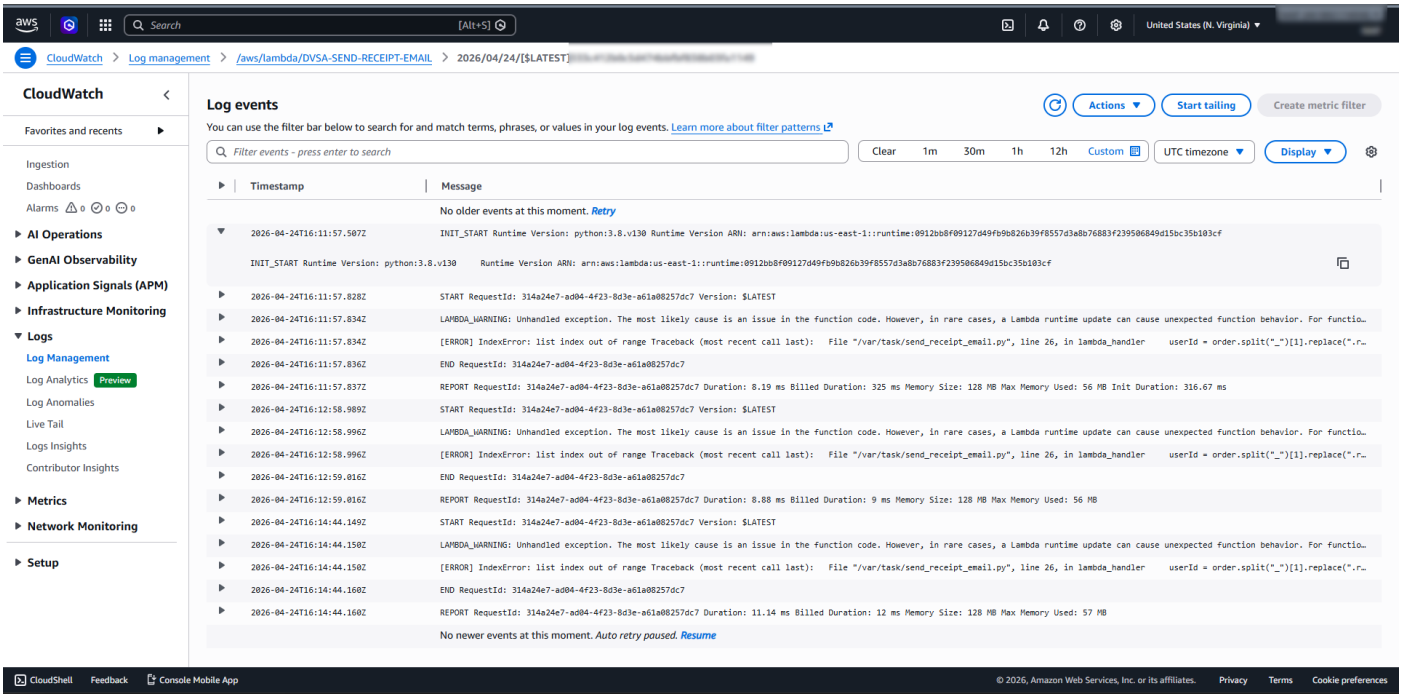

Figure 10: Screenshot #8 - Post-Fix CloudWatch No Credential Leak

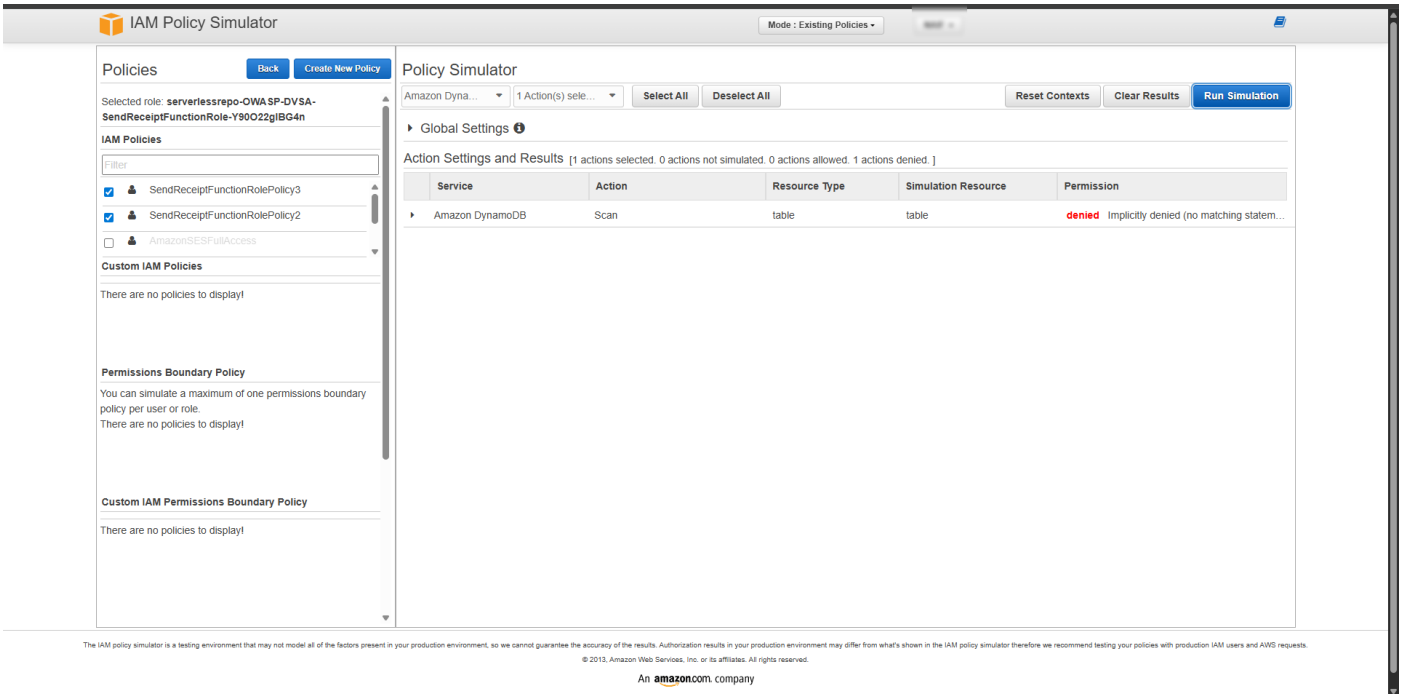

Figure 11: Screenshot #9 - IAM Simulator After Fix Denied

9. Structured Operation and Security Analysis

Intended Logic and Security Rule(s):

The **DVSA-SEND-RECEIPT-EMAIL** Lambda function is intended to process receipt-related events only. It should access only the minimum AWS resources required for receipt handling, and it should never expose runtime secrets or credentials in logs. Its execution role should follow the principle of least privilege and should not permit unrelated DynamoDB enumeration or broad backend data access.

Evidence Sources and Behavior Trace:

The analysis relied on the Lambda execution role policies, the **send_receipt_email.py** function code, the CloudWatch log output after the S3 trigger, and the IAM Policy Simulator results before and after the fix. The S3 upload triggered the Lambda, the function executed, and CloudWatch logs showed that sensitive environment variables were exposed when debugging output was added.

Normal vs. Exploit Behavior:

Under normal behavior, the function should process receipt events without disclosing credentials and without possessing unnecessary database permissions. Under exploit conditions, the function leaked temporary AWS credentials into CloudWatch logs, and its execution role had broader DynamoDB permissions than required, increasing the blast radius if the function were abused.

Why This Is a Security Deviation / Fix / Verification:

This is a security deviation because a receipt-processing Lambda function should not expose runtime credentials and should not have excessive database permissions unrelated to its task. The fix removed unsafe credential logging and reduced **SendReceiptFunctionRolePolicy2** to least privilege. Verification showed that sensitive AWS credential fields no longer appeared in CloudWatch logs and that **dynamodb:Scan** was denied after the policy update.

10. Takeaway / Lessons Learned

This lesson shows that serverless security is not only about preventing code flaws, but also about correctly limiting IAM permissions and avoiding unsafe logging practices. Even a small debugging change can become dangerous if it exposes credentials in logs. Applying least privilege and reviewing what a function really needs are essential to reducing risk in AWS serverless applications.